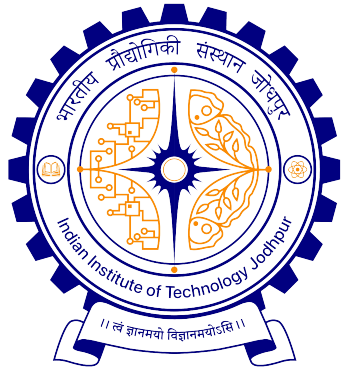




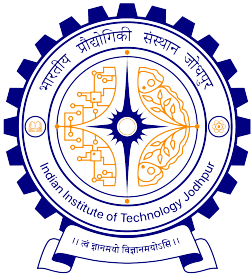
Introduction to Computer Science (CSL1010)

C Debugging and Tracing L10

Suchetana Chakraborty

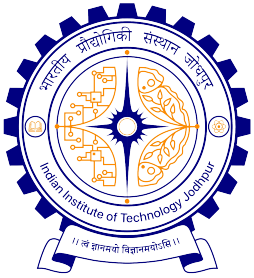
Department of Computer Science, IIT Jodhpur

Spring, 2026



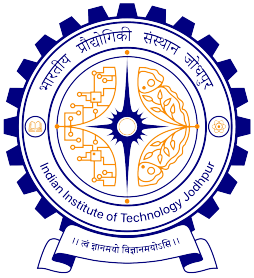
Bugs in a code

- A software bug is a problem causing a program to crash or produce invalid output
 - Most of the time, bugs are due to human errors in source code or in the system design
- The term bug was used in an account by computer pioneer Grace Hopper in 1946, while working at the computing laboratory at Harvard University in two computers with names Mark II and Mark III
 - Operators traced an error in the Mark II to a moth trapped in a relay, coining the term bug Source:
- Since then, the term debug is used to name the process of detecting and correcting software defects (or bugs) in a system



Logging

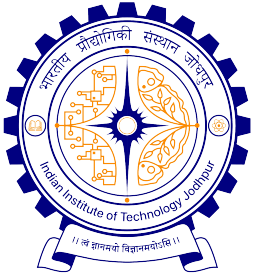
- Logging (i.e., inserting print statements or using logging libraries to output information about the program's state at various points in the execution) is commonly used by programmers to debug their programs



Debugging

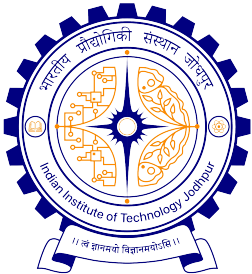
What can debuggers do?

- Run programs
- Make the program stops on specified places or on specified conditions
- Give information about current variables' values, the memory and the stack
- Let you examine the program execution step by step - *stepping*
- Let you examine the change of program variables' values - *tracing*



Debugging - GDB

- GDB (GNU Debugger) is a popular debugger tool that runs on Unix-like systems and works for different programming languages, including C, C++, or Go, among others
 - GDB was first written by Richard Stallman in 1986 as part of the GNU project
- GDB offers different features for tracing the execution of computer programs, such as:
 - Step by step execution
 - Monitor and modify the values of programs' internal variables
 - Call functions independently of the program's normal behavior



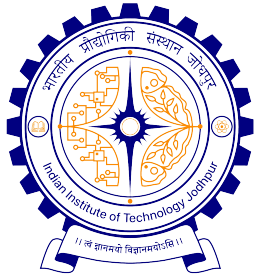
GDB

- To debug a C program with GDB, first, the program should be compiled including the `-g` flag (creates the symbol table)
 - This flag is used to include debugging information (known as debug symbols) in the compiled executable
 - Those debug symbols are required by debugging tools like GDB to map the blocks of binary operations onto the original source code in the step-by-step execution

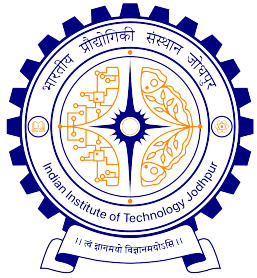
```
$ gcc -g program.c -o program
```

- Then, we need to invoke the command `gdb` using the program name as argument:

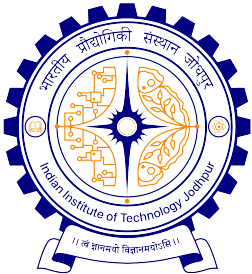
```
$ gdb program
```



GDB Commands - I



GDB Commands - II



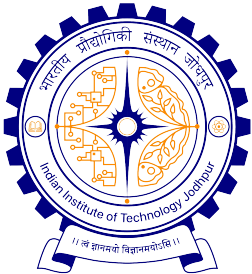
GDB - Running Program

`run` (or `r`) -- creates an inferior process that runs your program.

- if there are no execution errors the program will finish and results will be displayed
- in case of error, the GDB will show:
 - the line the program has stopped on and
 - a short description of what it believes has caused the error

There is a certain information that affects the execution of a program:

- program's arguments
- program's environment
- program's working directory
- the standard input and output



GDB – Breakpoints and Watchpoints

- Breakpoints and watchpoints allow you to specify the places or the conditions where you want your program to stop.

break *arg* – stops when the execution reaches the specified line

/ *arg* – function name, line number, +/- offset /

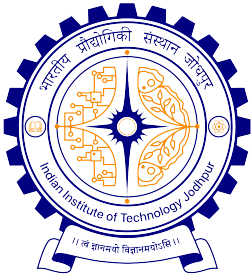
watch *expr* – stops whenever the value of the expression changes

clear [*arg*]

Without arguments deletes any breakpoint at the next instruction to be executed in the current stack frame

delete [*bnum*]

Without arguments deletes all breakpoints.



Examining Variable States

! Global variables can be examined from every point in the source file.

! Local variables – can be examined only in their scope or using:

file::variable or *function::variable*

The variable type: **ptype** *var*

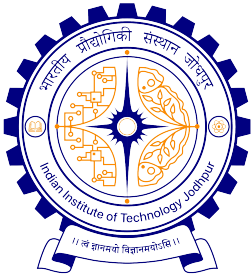
Current value: **print** *var*

Automatic display: **display** *var* – adds *var* to the *automatic display list*.

undisplay *dnum*

Specifying the output format (x, o, d, u, t, a, f, and c):

print /t *var* - prints the value of *var* in binary format



GDB – Value History

The *value history* keeps the values printed by the `print` command.

Previously printed values can be accessed by typing \$ followed by their history number.

\$ - refers to the most recent value and

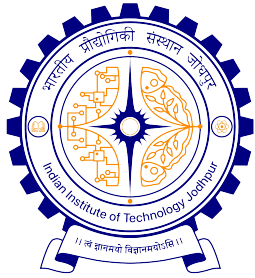
\$\$*n* - refers to the *n*-th value from the end.

`show values [n|+]`

Without argument – the last 10 values.

n – 10 values centered around *n*

+ – 10 values after the last printed



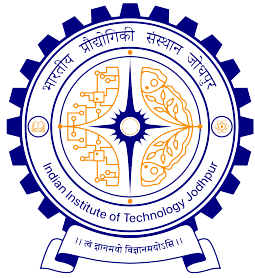
GDB – Stepping through the Program

step [*count*] – program execution continue to next source line going into function calls.

next [*count*] – program execution continue to the next source line omitting function calls.

continue – resume program execution

until – continue until the next source line in the current stack frame is reached. /useful to exit from loops/



GDB – Altering Execution

Returning from a function

finish – forced return

return [*ret_value*] – pops the current stack frame

Continuing at different address

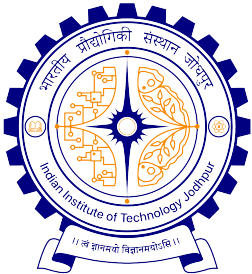
jump *line_num* | **address*

Altering the value of a variable

set *i=256*

Proceeding to a specified point:

until [*line_num* | **address* | *function_name*]



GDB – Stack Frame

- Stack frames are identified by their addresses, which are kept in the *frame pointer* register.

□ Selecting a frame:

frame *n* | *addr*

up *n*

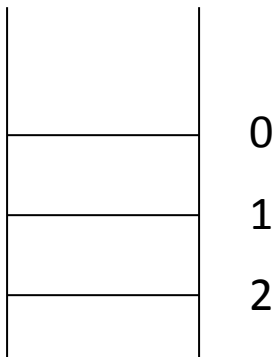
down *n*

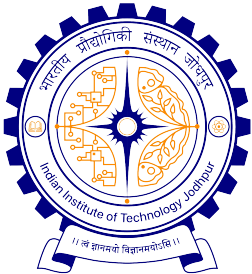
□ Information about the current frame

frame – brief description

info args – shows function arguments

info locals – shows local variables





GDB – Convenience Variable

- Convenience variables are used to store values that you may want to refer later. Any string preceded by `$` is regarded as a convenience variable.

Ex.: `$table = *table_ptr`

- There are several automatically created convenience variables:

`$pc` – program counter

`$sp` – stack pointer

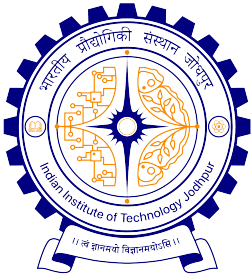
`$fp` – frame pointer

`$ps` – processor status

`$__` – contains the last examined address

`$__` – the value in the last examined address

`$_exitcode` – the exit code of the debugged program



Example: Debugging with GDB

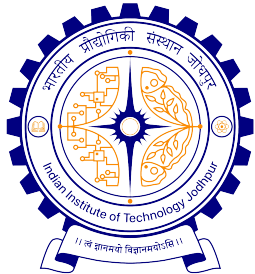
```
#include <stdio.h>

void printArray(int *arr, int size) {
    for (int i = 0; i <= size; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}

int main() {
    int numbers[] = { 10, 20, 30, 40, 50 };
    int size = sizeof(numbers) /
sizeof(numbers[0]);
    printf("Array contents:\n");
    printArray(numbers, size);
    return 0;
}
```

Output:

```
Array contents:
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
arr[5] = 0
```



Example: Debugging with GDB

1. Compile the program with debugging information

```
gcc -g debug_gdb.c -o debug_gdb
```

2. Launch GDB with the compiled executable:

```
gdb ./debug_gdb
```

3. Set a breakpoint:

```
break 5
```

4. Run the program:

```
run
```

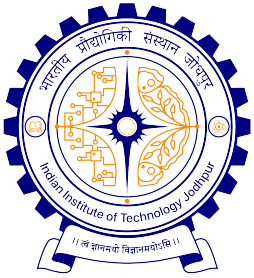
5. Inspect variables:

```
print size
```

6. Step through the code:

```
next
```

```
continue
```



Resources

- https://www.tutorialspoint.com/gnu_debugger/index.htm
- <https://sourceware.org/gdb/current/onlinedocs/gdb/>